

2026

Розробка програмної системи для візуалізації, порівняння та аналізу ефективності алгоритмів сортування засобами Java

Комп'ютерні науки

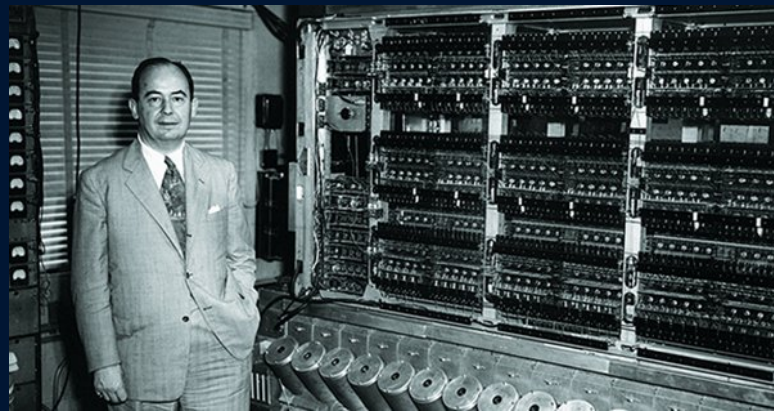
Виконав - Яровий Богдан, КНД-42

Керівник - Березовська Юлія Володимирівна

Сортування даних

«Сортування та пошук — це серцебиття комп'ютера. Усе інше просто перекачує кров з місця на місце»

Дональд Эрвин Кнут



У 1945 році Джон фон Нейман створив першу в історії програму для ЕОМ, і її суттю було сортування великих об'ємів даних.

Дві причини, чому сортування важливе для інтелектуальних технологій



Прискорення пошуку інформації

Швидко знайти потрібний елемент у несортованому масиві можна лише повним перебором, що займає багато часу. У сортованих даних можна використовувати ефективні алгоритми, наприклад, бінарний пошук, який знаходить інформацію майже миттєво навіть серед мільярдів записів

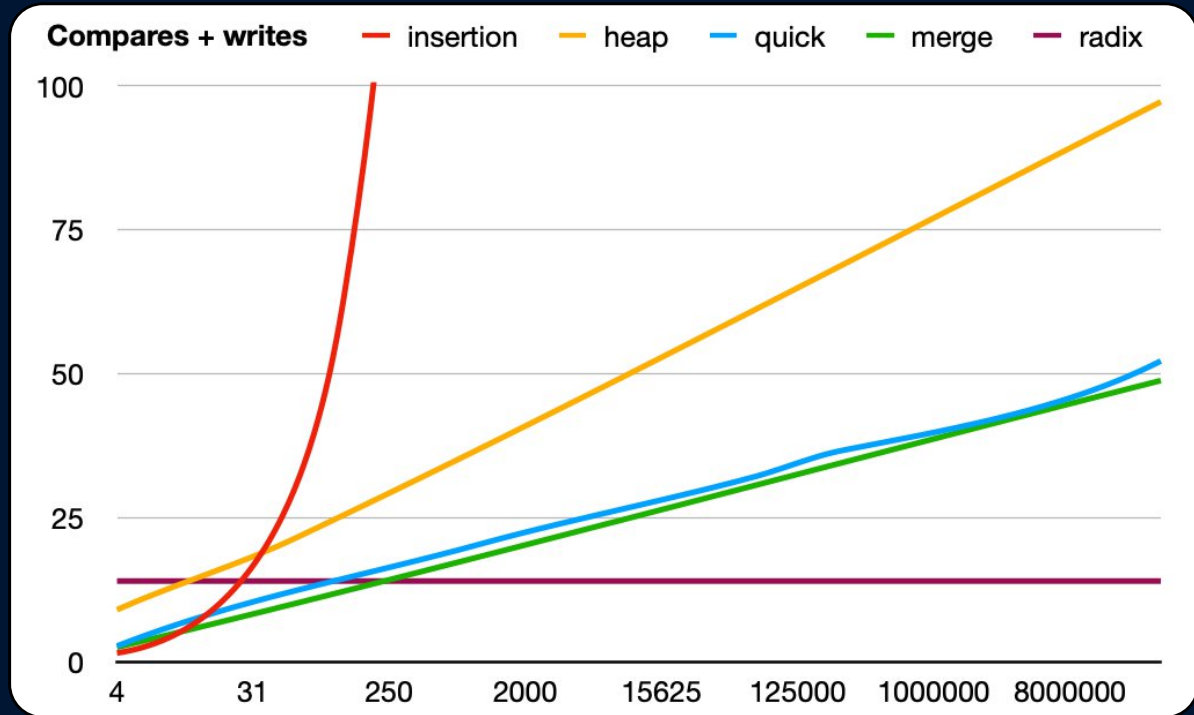


Спрощення аналізу та візуалізації

Впорядковані дані набагато легше аналізувати як людині, так і комп'ютерним програмам. Сортування дозволяє швидко виявляти екстремальні значення (мінімуми та максимуми), групувати схожі елементи, знаходити дублікати, вираховувати медіану та будувати зрозумілі графіки чи звіти.

Найвідоміші алгоритми сортування

- Insertion Sort
або
Bubble Sort
- Heap Sort
- Quick Sort
- Merge Sort
- Radix Sort



В чому проблема?

Алгоритми сортування
використовується в 99.9%
усіх систем у світі.

Знання таких алгоритмів є
must have для кожного
розробника в ІТ.

Зрозуміти такі алгоритми
складно - бо вони складні та
абстрактні.

Що робити?

Краще один раз **побачити**
ніж сто разів почути.



Існує безліч рішень, але є нюанс..

В інтернеті вистачає різноманітних ілюстраторів алгоритмів сортування. Від приватних рішень на GitHub, до публічних web-сайтів.

Головна проблема таких рішень - алгоритми жорстко заcodedовані. Не можна додати свій власний алгоритм.

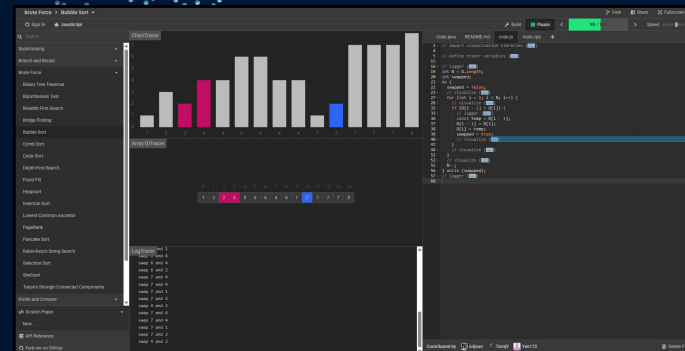


Виняток чи exception?

Algorithm Visualizer - web-ресурс для демонстрації роботи алгоритмів.

Його перевага перед подібними ресурсами в тому, що він надає можливість програмувати власні алгоритми, і не тільки алгоритми сортування. Писати код можна на мовах: Java, JavaScript, C++.

Але не все так однозначно...

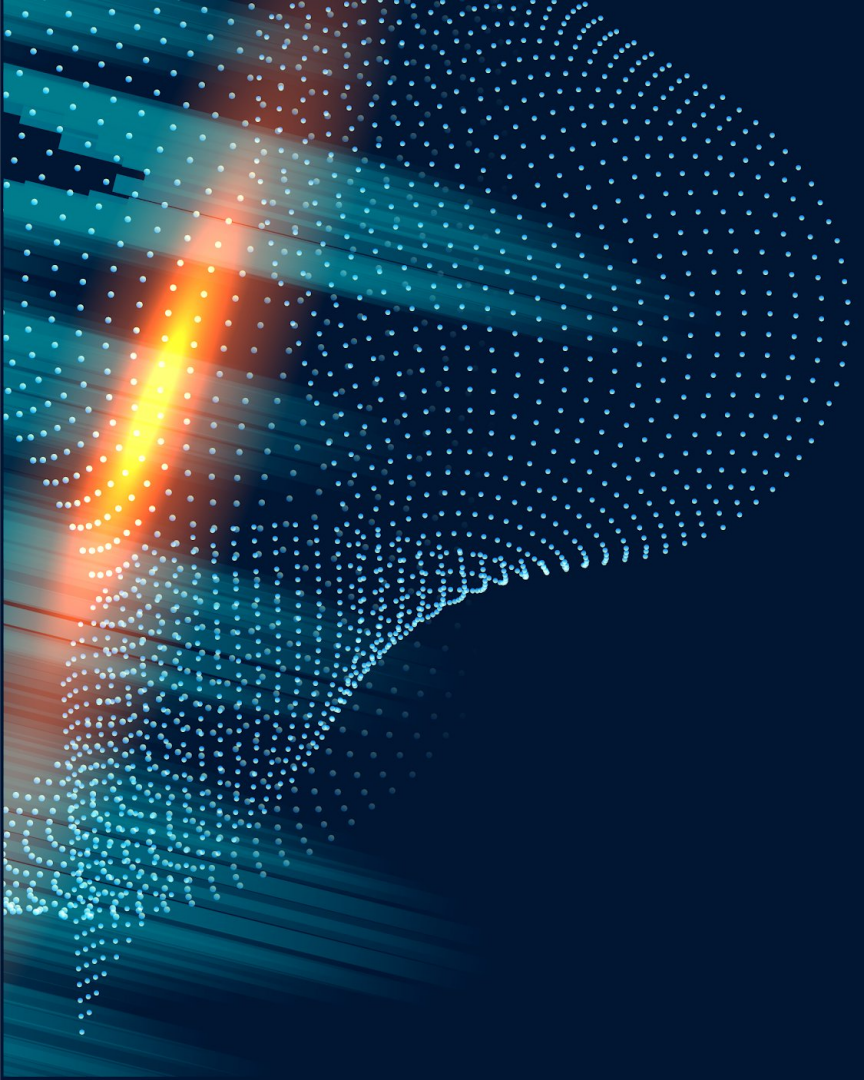


```
1 import org.algorithm_visualizer.*;
2
3 import java.util.Arrays;
4
5 public class Main {
6
7     private static ChartTracer chartTracer = new ChartTracer();
8     private static LogTracer logTracer = new LogTracer("Console");
9
10    private static Integer [] array = (Integer[]) new Randomize.ArrayID(15, new Randomize.Integer(1, 20)).create();
11
12    public static void main(String[] args) {
13        int length = array.length;
14
15        logTracer.printf("Original array = %s\n", Arrays.toString(array));
16        chartTracer.set(array);
17        Layout.setRoot(new VerticalLayout(new Commander[] {chartTracer, logTracer}));
18        Tracer.delay();
19
20        boolean flag;
21
22        for (int i = length - 1; i > 0; i--) {
23            flag = true;
24            for (int j = 0; j < i; j++) {
25                chartTracer.select(j);
26                chartTracer.select(j + 1);
27                Tracer.delay();
28                if (array[j] > array[j + 1]) {
29                    logTracer.printf("swap %s and %s\n", array[j], array[j + 1]);
30                    swap(j, j + 1, array);
31                    flag = false;
32                }
33                chartTracer.deselect(j);
34                chartTracer.deselect(j + 1);
35            }
36            if (flag) {
37                break;
38            }
39        }
40
41        logTracer.printf("sorted array = %s\n", Arrays.toString(array));
42
43    }
44
45    private static void swap(int x, int y, Integer [] array) {
46
47    }
48
49 }
```

Якщо подивитись
ближче, то можна
зрозуміти, що код
робить все:

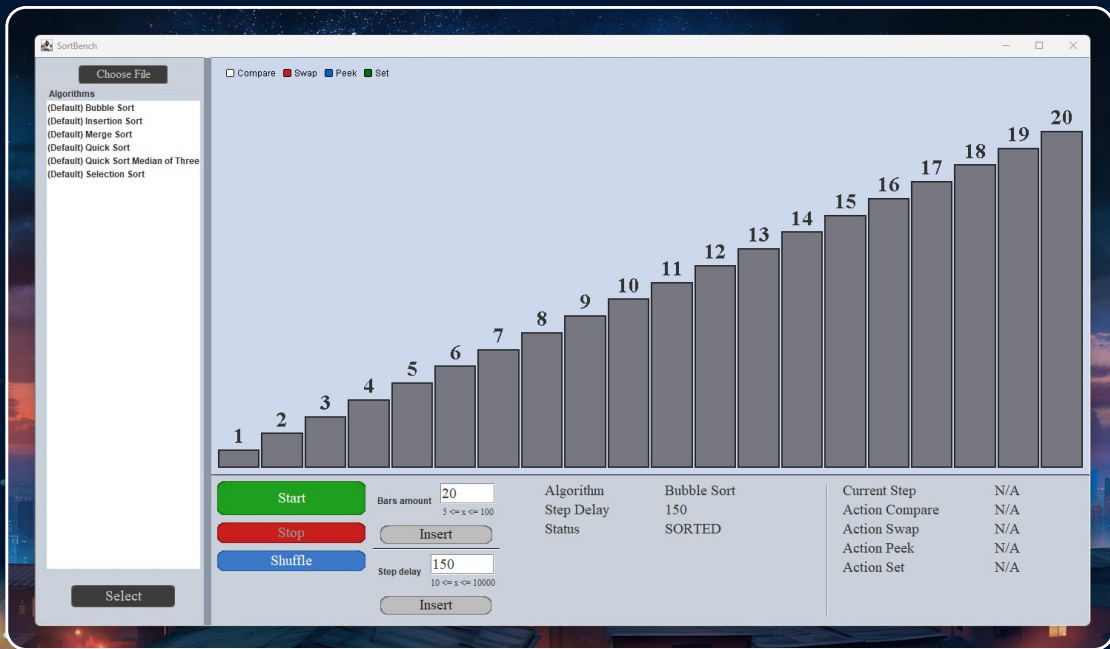
- бізнес-логіку;
- генерацію даних;
- відображення
елементів;
- логування.

```
1 import org.algorithm_visualizer.*;
2
3 import java.util.Arrays;
4
5 public class Main {
6
7     private static ChartTracer chartTracer = new ChartTracer();
8
9     private static LogTracer logTracer = new LogTracer("Console");
10
11     private static Integer [] array = (Integer[]) new Randomize.ArrayID(15, new Randomize.Integer(1, 20)).create();
12
13     public static void main(String[] args) {
14
15         int length = array.length;
16
17         logTracer.printf("original array = %s\n", Arrays.toString(array));
18         chartTracer.set(array);
19         Layout.setRoot(new VerticalLayout(new Commander[]{chartTracer, logTracer}));
20         Tracer.delay();
21
22         boolean flag;
23
24         for (int i = length - 1; i > 0; i--) {
25             flag = true;
26             for (int j = 0; j < i; j++) {
27                 chartTracer.select(j);
28                 chartTracer.select(j + 1);
29                 Tracer.delay();
30                 if (array[j] > array[j + 1]) {
31                     logTracer.printf("swap %s and %s\n", array[j], array[j + 1]);
32                     swap(j, j + 1, array);
33                     flag = false;
34                 }
35                 chartTracer.deselect(j);
36                 chartTracer.deselect(j + 1);
37             }
38             if (flag) {
39                 break;
40             }
41         }
42
43         logTracer.printf("sorted array = %s\n", Arrays.toString(array));
44     }
45
46     private static void swap(int x, int y, Integer [] array) {
47
48     }
49 }
```



SORT BENCH

Застосунок - який може



Sort Bench - це програмний засіб, який створений для візуалізації, аналізу та розробки алгоритмів сортування.

Головний принцип - просто та зрозуміло.

Головні функції

1. Відображення та візуалізація переміщення елементів колекції
2. Вибір алгоритмів сортування
3. Налаштування умов сортування
4. Можливість додати власний алгоритм
5. Метрики операцій



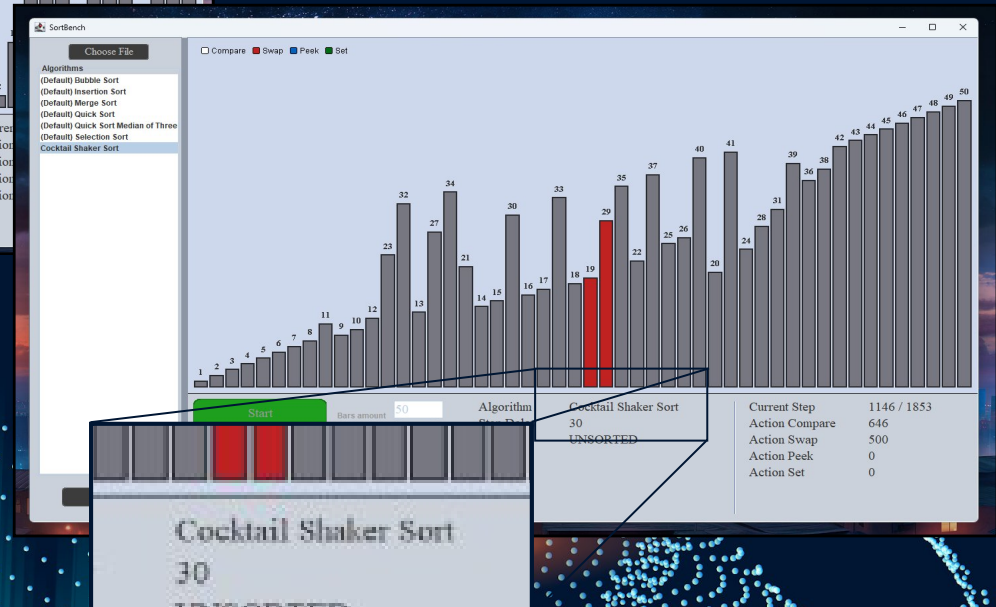
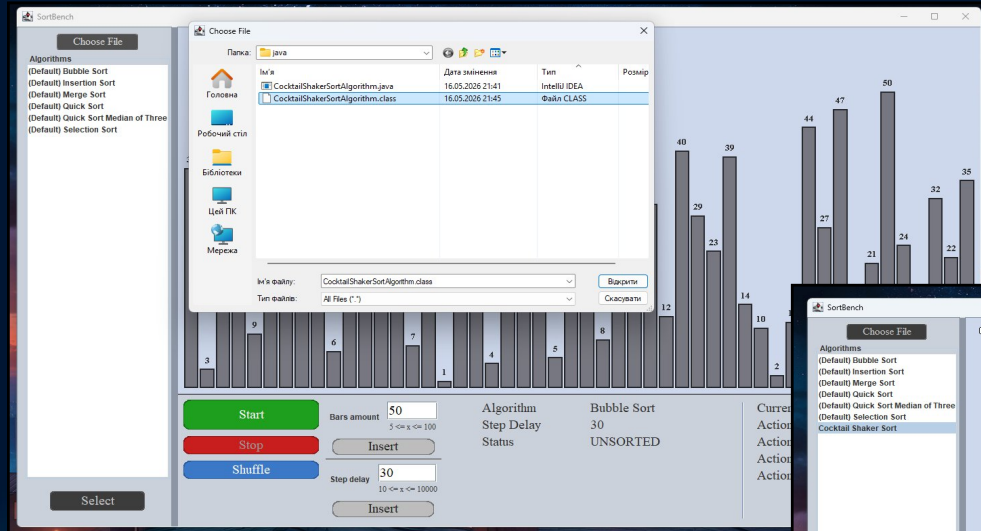
Record-and-replay підхід

Для відображення роботи алгоритму сортування було впроваджено record-and-replay підхід:

- Користувач натискає кнопку **Start**
- Елементи передаються алгоритму сортування, де кожна взаємодія з колекцією елементів зберігається
- Вікно з елементами повторює події



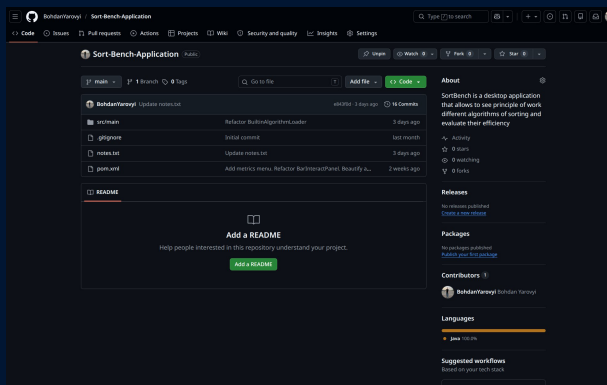
Додавання власного алгоритму



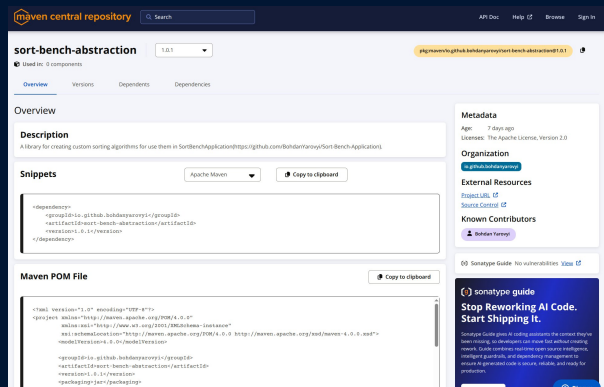
Sort Bench Abstraction

Sort-Bench-Abstraction - це легковісна бібліотека, яка потрібна для створення алгоритмів, що можна використовувати в Sort Bench.

Бібліотека доступна на таких платформах



GitHub



Maven Central Repository

Roadmap по запуску свого алгоритму сортування в Sort Bench



Код алгоритму Bubble Sort

Sort Bench

```
public class BubbleSortAlgorithm extends SortAlgorithm {
    private String name = "Bubble Sort";

    @Override
    public void sort(SortingCollection collection) {
        int size = collection.getSize();
        for (int i = 0; i < size; i++) {
            for (int j = 0; j < size - 1 - i; j++) {
                if (collection.compare(j, j + 1) > 0) {
                    collection.swap(j, j + 1);
                }
            }
        }
    }

    @Override
    public String getName() {
        return name;
    }

    @Override
    public void setName(String name) {
        this.name = name;
    }
}
```

Algorithm Visualizer

```
public class Main {

    private static ChartTracer chartTracer = new ChartTracer();
    private static LogTracer logTracer = new LogTracer("Console");

    private static Integer [] array = (Integer[]) new Randomize.Array1D(15, new Randomize.Integer(1, 20)).create();

    public static void main(String[] args) {

        int length = array.length;

        logTracer.print("original array = %s\n", Arrays.toString(array));
        chartTracer.set(array);
        Layout.setRoot(new VerticalLayout(new Commander[] {chartTracer, logTracer}));
        Tracer.delay();

        boolean flag;

        for (int i = length - 1; i > 0; i--) {
            flag = true;
            for (int j = 0; j < i; j++) {
                chartTracer.select(j);
                chartTracer.select(j + 1);
                Tracer.delay();
                if (array[j] > array[j + 1]) {
                    logTracer.print("swap %s and %s\n", array[j], array[j + 1]);
                    swap(j, j + 1, array);
                    flag = false;
                }
                chartTracer.deselect(j);
                chartTracer.deselect(j + 1);
            }
            if (flag) {
                break;
            }
        }

        logTracer.print("sorted array = %s\n", Arrays.toString(array));
    }

    private static void swap(int x, int y, Integer [] array) {
        int temp = array[x];
        array[x] = array[y];
        array[y] = temp;
        chartTracer.patch(x, array[x]);
        chartTracer.patch(y, array[y]);
        Tracer.delay();
        chartTracer.depatch(x);
        chartTracer.depatch(y);
    }
}
```

Висновки

- Розроблено програмну систему SortBench для візуалізації алгоритмів сортування
- Реалізовано 6 алгоритмів сортування з покроковою візуалізацією
- Розроблено та опубліковано бібліотеку SortBenchAbstraction на Maven Central
- Реалізовано механізм завантаження власних алгоритмів без модифікації застосунку
- Проведено тестування — всі функції працюють коректно



Робота Sort Bench

